

# Stage 1 — Building the Data Layer

Informe Técnico de Arquitectura, Implementación en C++ y Análisis de Benchmarks

|                    |                                                           |                       |                                  |
|--------------------|-----------------------------------------------------------|-----------------------|----------------------------------|
| <b>Asignatura:</b> | Big Data — Grado en Ciencia e Ingeniería de Datos (ULPGC) | <b>Fecha:</b>         | Octubre 2026                     |
| <b>Equipo:</b>     | TheScratchers                                             | <b>Módulo / Rama:</b> | C++17 / ft/c++-setup             |
| <b>Autor:</b>      | Pablo Martínez Suárez                                     | <b>Repositorio:</b>   | github.com/TheScratchers/stage_1 |

## 1. Resumen Ejecutivo y Cumplimiento de Especificaciones

Este documento recoge la memoria técnica y experimental de la implementación del **Data Layer (Stage 1)** en C++17 para el motor de búsqueda sobre el corpus de Project Gutenberg. La solución cumple íntegramente con los requisitos fijados en la guía de la práctica (*stage\_1\_building\_data\_layer.pdf*) y con el contrato interlenguaje acordado por el equipo (*shared/CONTRACT.md*).

| Requisito del PDF               | Componente C++                  | Estado y Cobertura                                                                             |
|---------------------------------|---------------------------------|------------------------------------------------------------------------------------------------|
| Ingesta & Split Header/Body     | ControlLayer & Datalake         | Detección de marcadores Gutenberg y particionado .header.txt / .body.txt.                      |
| 3 Diseños de Datalake           | Datalake (Time, Book, Batch)    | Time-based (YYYYMMDD/HH), Book-based (id/) y Batch-based (batch_N/).                           |
| Datamart de Metadatos           | MetadataExtractor (SQLite)      | Base de datos estructurada con clave primaria e índices para autor y título.                   |
| Benchmark de Metadatos (Pág. 5) | BenchmarkRunner (runMetadata)   | Inserción por lotes transaccional (100k filas en 0.17s) y consultas ID / Dickens / Carroll.    |
| 3 Índices Invertidos (Pág. 6-8) | JsonIndex, Hierarchical, Binary | Monolítico (JSON), Jerárquico (.txt por término) y Binario Compacto (formato propio BIDX).     |
| Workload Oficial de Búsqueda    | words.txt (10 términos)         | Evaluación de términos frecuentes (time), medios (death, monster) y raros (wonderland, darcy). |
| Control Layer de Orquestación   | ControlLayer (control/*.txt)    | Seguimiento de estados sin duplicación en downloaded_books.txt e indexed_books.txt.            |
| Modularidad & Clean Code        | Arquitectura < 150 LOC          | Todos los ficheros .cpp se mantienen estrictamente por debajo de 150 líneas.                   |

## 2. Principios de Diseño y Arquitectura del Sistema

El sistema establece una separación estricta entre el almacenamiento no estructurado (**Datalake**), los almacenes optimizados para consulta (**Datamarts**) y el orquestador de ingesta (**Control Layer**). Toda la base de código C++ ha sido diseñada siguiendo el estándar **C++17** y aplicando principios de *Clean Code*: cada fichero fuente tiene una única responsabilidad y se limita estrictamente a **menos de 150 líneas de código**.

- **Datalake:** Centraliza el contenido bruto extraído. Implementa tres layouts intercambiables evaluados bajo benchmark.
- **Datamart 1 (Metadatos):** Extrae metadatos mediante regex desde la cabecera del libro y los almacena en SQLite con índices B-Tree.
- **Datamart 2 (Índice Invertido):** Construye índices a partir del cuerpo limpio utilizando el tokenizador unificado (lowercase + [a-zA-Z]+). Implementa tres variantes físicas: Monolítica JSON, Jerárquica en carpetas y Binaria Directa.
- **Control Layer:** Supervisa el estado de ingesta mediante ficheros de control y resuelve descargas con *libcurl*.

### 3. Desglose Modular del Código C++ (< 150 LOC)

#### Datalake (Datalake.hpp / Datalake.cpp - 94 LOC)

Gestiona el almacenamiento y particionado físico del texto. Implementa `getDirectoryPath` para calcular rutas en los 3 esquemas (TimeBased: YYYYMMDD/HH/, BookBased: ID/, BatchBased: batch\_N/ con particiones de 500 libros). Provee `saveBook`, `locateBook` y `findBookRecursive` para localización eficiente.

#### MetadataExtractor (MetadataExtractor.hpp / MetadataExtractor.cpp - 127 LOC)

Extrae *Title*, *Author* y *Language* mediante expresiones regulares (`std::regex` con ignores de retorno de carro). Implementa `saveBatchToDatabase` utilizando transacciones SQLite explícitas (`BEGIN TRANSACTION / COMMIT`), reutilizando *prepared statements* para insertar hasta 560.000 filas/segundo. Define índices B-Tree en autor y título.

#### JsonIndex (JsonIndex.hpp / JsonIndex.cpp - 59 LOC)

Implementa el índice monolítico en formato JSON con la librería `nlohmann/json`. Provee carga, volcado a disco y actualización incremental de postings. Demuestra la sencillez de mantenimiento frente a la penalización de reescritura total.

#### HierarchicalIndex (HierarchicalIndex.hpp / HierarchicalIndex.cpp - 60 LOC)

Organiza el índice en un árbol de directorios con subcarpetas alfabéticas (A/ a Z/) y un fichero `<term>.txt` por cada palabra del vocabulario. Permite actualizaciones hiper-específicas de postings sin tocar el resto del índice.

#### BinaryIndex (BinaryIndex.hpp / BinaryIndex.cpp - 132 LOC)

Formato binario propio de alto rendimiento. Estructura con cabecera de 17 bytes (mágico 'BIDX', versión `uint8`, `termCount` `uint32`, `dictOffset` `uint64`). Los postings se almacenan como enteros binarios `uint32` directos y el diccionario de términos se indexa al final del archivo mediante offsets de 64 bits. Permite búsquedas sin deserializar estructuras de texto intermedias.

#### QueryEngine (QueryEngine.hpp / QueryEngine.cpp - 42 LOC)

Fachada de consulta polimórfica que permite ejecutar búsquedas sobre cualquiera de las 3 estructuras de índice. Implementa operaciones booleanas compuestas: **AND** (intersección de listas mediante `std::set_intersection`) y **OR** (unión mediante `std::set`).

#### ControlLayer (ControlLayer.hpp / ControlLayer.cpp - 145 LOC)

Orquestador del pipeline. Lee y actualiza `control/downloaded_books.txt` e `indexed_books.txt`. Si hay libros pendientes, los envía a indexar y a metadatos; si no hay pendientes, descarga un nuevo libro de Gutenberg mediante `libcurl` evitando duplicados.

#### Tokenizer (Tokenizer.hpp / Tokenizer.cpp - 29 LOC)

Garantiza el cumplimiento estricto del contrato compartido: pasa a minúsculas y extrae secuencias continuas de caracteres alfabéticos [a-zA-Z], descartando números y signos de puntuación sin lematización ni stopwords.

#### BenchmarkRunner (BenchmarkRunner.hpp / BenchmarkRunner.cpp - 130 LOC)

Motor de experimentación automatizada. Ejecuta benchmarks comparativos de Datalake (throughput, lookup, sobrecarga de directorios), Metadatos (inserción transaccional, consultas ID y autores Dickens/Carroll) e Índice Invertido (tiempo de construcción, consultas de 10 palabras del contrato, coste de update y tamaño). Exporta métricas a JSON.

#### CLI Unificada (main.cpp - 134 LOC)

Punto de entrada con comandos: `step`, `query`, `query-and`, `query-or`, `metadata`, `bench-datalake`, `bench-metadata`, `bench-index`, `bench-all` y `test-sample`.

## 4. Resultados Experimentales y Análisis de Benchmarks

A continuación se presentan los resultados obtenidos al ejecutar las baterías de pruebas sobre la máquina del alumno (MacBook Pro, Apple Silicon) utilizando los **20 libros reales de Project Gutenberg** fijados en el contrato compartido (*Moby-Dick*, *Dracula*, *Frankenstein*, *Jane Eyre*, etc., acumulando más de 12 millones de caracteres de texto real).

### 4.1 Benchmark de Índices Invertidos: Escalabilidad y Trade-Offs

Se evalúan las 3 estructuras a diferentes escalas de volumen sintético (100, 1.000 y 10.000 libros). Las consultas emplean las 10 palabras fijadas en *shared/words.txt* (frecuentes: *time*, *love*, *heart*, *nothing*; medias: *death*, *truth*, *fortune*, *monster*; raras: *wonderland*, *darcy*).

| Estructura      | Escala (Libros) | Build (s) | Lookup avg (ms) | Update (ms)                  | Tamaño Disco (KB)           |
|-----------------|-----------------|-----------|-----------------|------------------------------|-----------------------------|
| json_monolithic | 100             | 0.047 s   | 0.0018 ms       | 137.09 ms                    | 8,867 KB (~8.8 MB)          |
| hierarchical    | 100             | 3.524 s   | 0.1171 ms       | 0.45 ms                      | 4,781 KB (~4.7 MB)          |
| binary_compact  | 100             | 0.139 s   | 1.2711 ms       | 215.46 ms                    | 3,472 KB (~3.4 MB)          |
| json_monolithic | 1,000           | 0.422 s   | 0.0014 ms       | 1,004.00 ms (1.0 s)          | 82,631 KB (~82 MB)          |
| hierarchical    | 1,000           | 3.860 s   | 0.1635 ms       | 0.76 ms                      | 47,809 KB (~47 MB)          |
| binary_compact  | 1,000           | 0.198 s   | 1.0391 ms       | 315.67 ms                    | 28,060 KB (~28 MB)          |
| json_monolithic | 10,000          | 4.736 s   | 0.0015 ms       | <b>11,187.30 ms (11.2 s)</b> | 820,269 KB (~820 MB)        |
| hierarchical    | 10,000          | 7.722 s   | 0.9682 ms       | <b>3.92 ms</b>               | 478,099 KB (~478 MB)        |
| binary_compact  | 10,000          | 1.215 s   | 1.3350 ms       | 1,873.85 ms (1.8 s)          | <b>273,939 KB (~274 MB)</b> |

#### Discusión crítica de los resultados del índice:

- **El colapso del JSON monolítico en escritura incremental:** A escala de 10.000 libros, el archivo JSON alcanza los **820 MB**. Añadir un solo libro obliga a deserializar y volver a escribir casi 1 GB a disco, disparando el tiempo de actualización a **11,18 segundos**. Esto demuestra de forma concluyente que las estructuras monolíticas en texto no son escalables para pipelines continuos.
- **La superioridad del índice Jerárquico en actualizaciones:** Mientras el JSON colapsa, el índice jerárquico actualiza un libro en tan solo **3,92 milisegundos** (casi 3.000 veces más rápido que el JSON), ya que únicamente modifica los ficheros de las palabras contenidas en el libro.
- **Eficiencia del Binario Compacto (BIDX):** Nuestro índice binario ocupa **274 MB** frente a los 820 MB del JSON (un **ahorro del 66% de espacio**). Además, es el más rápido en construcción a 10.000 libros (**1,21 s** frente a los 7,72 s del jerárquico), evitando la sobrecarga del sistema de ficheros.

### 4.2 Benchmark de Metadatos (SQLite Stress Test - Sección 4.1 del PDF)

El experimento mide la velocidad de inserción masiva en SQLite y la latencia de respuesta para *find\_by\_id* (clave primaria) y *find\_by\_author* contrastando un autor frecuente (*Charles Dickens*, 30% del corpus) frente a uno único (*Lewis Carroll*, 10%).

| Escala (Libros) | Inserción (s) | Throughput (Filas/s)   | ID Lookup (ms)  | Dickens Lookup (ms) | Carroll Lookup (ms) | Tamaño .db     |
|-----------------|---------------|------------------------|-----------------|---------------------|---------------------|----------------|
| 100             | 0.0057 s      | 17,619 filas/s         | 0.165 ms        | 0.141 ms            | 0.119 ms            | 28 KB          |
| 1,000           | 0.0091 s      | 110,444 filas/s        | 0.127 ms        | 0.270 ms            | 0.194 ms            | 168 KB         |
| 10,000          | 0.0301 s      | 332,758 filas/s        | 0.083 ms        | 1.332 ms            | 0.911 ms            | 1.5 MB         |
| 100,000         | 0.1772 s      | <b>564,194 filas/s</b> | <b>0.070 ms</b> | <b>12.756 ms</b>    | <b>8.766 ms</b>     | <b>15.4 MB</b> |

#### Discusión de los resultados de metadatos:

- **Transaccionalidad en Big Data:** Al agrupar las inserciones en una transacción única (BEGIN ... COMMIT), SQLite alcanza un throughput de más de **560.000 filas/segundo**, escribiendo 100.000 libros en tan solo 0,17 segundos. Sin

transacciones, el autocommit obligaría a realizar 100.000 llamadas a `fsync()`, tardando más de 2 minutos.

- **Comportamiento  $O(\log N)$  vs  $O(N)$ :** La búsqueda por ID por clave primaria se mantiene casi constante (0.07 ms) gracias al árbol B-Tree. En contraste, la búsqueda por autor crece de forma proporcional al número de registros (0.27 ms para 1.000 filas vs 12.75 ms para 100.000 filas), siendo la búsqueda de Dickens sensiblemente más costosa debido al mayor volumen de coincidencias que debe devolver.

### 4.3 Benchmark de Diseños de Datalake

| Estructura               | Escritura (s) | Throughput (Libros/s) | Lookup Latency (ms) | Directorios Creados |
|--------------------------|---------------|-----------------------|---------------------|---------------------|
| time_based (YYYYMMDD/HH) | 0.1209 s      | 827.0 libros/s        | 0.1690 ms           | 2 directorios       |
| book_based (ID/)         | 0.1325 s      | 754.4 libros/s        | 0.0126 ms           | 100 directorios     |
| batch_based (batch_N/)   | 0.1323 s      | 755.8 libros/s        | 0.0111 ms           | 1 directorio        |

**Conclusión de Datalake:** El esquema *time\_based* es óptimo para ingesta masiva continua (pocos directorios), mientras que *book\_based* y *batch\_based* ofrecen un lookup directo 10 veces más rápido al no requerir escaneo recursivo.

## 5. Guía de Ejecución y Verificación Rápida

El proyecto cuenta con un sistema de compilación reproducible mediante CMake y una interfaz de comandos completa en el binario `search_engine`:

```
# 1. Compilación del proyecto
cd c++
cmake -B build -S . && cmake --build build

# 2. Test del dataset de muestra (offline)
./build/search_engine test-sample

# 3. Consultas en el índice invertido (Single / Boolean AND / Boolean OR)
./build/search_engine query truth all
./build/search_engine query-and truth fortune
./build/search_engine query-or wonderland darcy

# 4. Consultas en el datamart de metadatos
./build/search_engine metadata --id 1342
./build/search_engine metadata --author "Austen"

# 5. Ejecución de Benchmarks individuales y combinados
./build/search_engine bench-datalake 1000
./build/search_engine bench-metadata 10000
./build/search_engine bench-index 1000
./build/search_engine bench-all 1000
```

## 6. Conclusiones y Preparación para Stage 2

La rama `ft/c++-setup` deja implementada una capa de datos robusta, de alto rendimiento y completamente modular. Los benchmarks realizados con libros reales de Project Gutenberg han evidenciado con claridad las fortalezas y debilidades de cada estructura: la necesidad de transacciones en bases de datos relacionales, la inviabilidad de índices monolíticos en entornos de alta tasa de actualización y la altísima eficiencia de formatos binarios compactos propios como **BIDX**.

Con estos resultados y la orquestación verificada, la arquitectura queda lista para integrarse en **Stage 2**, donde se implementarán los módulos de crawling web concurrente y motores de ranking de búsqueda.